

ADVANTEST®

ADVANTEST CORPORATION

ATL-51
Scramble Program
Reference Manual

MANUAL NUMBER 8256294-17

Applicable OS
ASX/U-51

Trademarks

The trademarks and registered trademarks used for this manual are described below.

Product names which are not listed below are trademarks or registered trademarks of their respective owners.

- ADVANTEST, ADVANsite, ATLworks, and ATL are trademarks of ADVANTEST Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Revision History

Rev.	Date	Notes
01	Feb 15/96	
02	Mar 18/97	
03	Jun 30/97	
04	Oct 30/97	
05	Mar 10/98	
06	May 8/98	
07	Feb 10/99	
08	Jun 30/99	
09	Oct 25/99	
10	Mar 3/00	
11	Dec 1/00	
12	Apr 10/01	
13	Aug 20/01	
14	Aug 9/02	
15	Nov 12/02	
16	May 15/03	
17	Jul 25/03	

Table of Contents

Preface	Preface-1
Prior Information	Preface-1
Related Manuals	Preface-12
 1. General Description	 1-1
1. 1 Scramble Program Purpose	1-1
 2. Program Overview	 2-1
2. 1 Program Settings	2-1
2. 2 Compilation Messages	2-3
 3. Handling Characters	 3-1
3. 1 Constants	3-1
3. 2 Special Characters	3-2
 4. Common Control Statements	 4-1
4. 1 Common Control Statements	4-1
4. 1. 1 SCRAM	4-1
4. 1. 2 RDX	4-1
4. 1. 3 AMAX	4-2
4. 1. 4 END	4-3
4. 1. 5 ARIRAM DEFINE	4-3
4. 1. 6 ARIRAM Data Setting Statements	4-5
 5. Exclusive Control Statements	 5-1
5. 1 Setting Decode and Address Topology (In Logical Notation)	5-1
5. 1. 1 EXPRESSION	5-1
5. 1. 2 Decode Topology (Decode Correction Statements)	5-1

5. 1. 3	ADDRESS	5-9
5. 1. 4	Address Topology (Address Correction Statements)	5-9
5. 2	Setting Addresses and Data (Individually)	5-12
5. 2. 1	X Address and Y Address/Data Setting Statement	5-12
5. 2. 2	ALINK	5-13
5. 2. 3	Address Bit Linkage Statements	5-14
5. 3	Combined Settings of One-to-one and Logical Description	5-16
5. 3. 1	REPLACE	5-16
6.	Compilation Statement	6-1
6. 1	Adding Revision Information to the Object File	6-1
6. 1. 1	%REMARK Statement	6-1
Appendix 1.	Scramble Program Example	A1-1
Appendix 2.	Logical Description Program Example ..	A2-1
Appendix 3.	Example Program which Uses One-to-one and Logical Description	A3-1
List of Figures		F-1
List of Tables		T-1
Index		I-1

Preface

Prior Information

References to the T5500 series apply to the T5593, T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P unless otherwise specified.

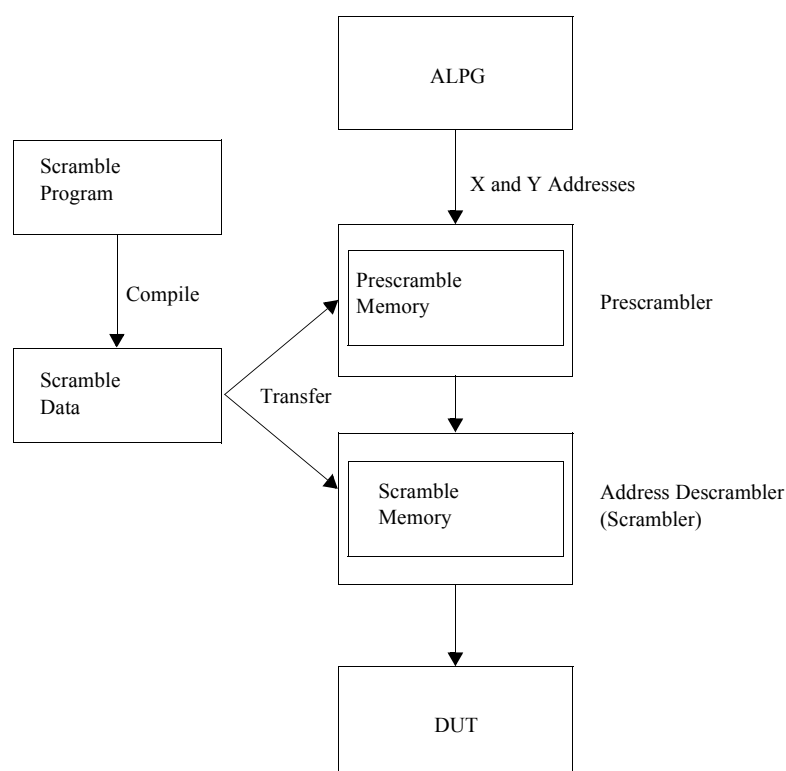
- Address conversion

The address descrambler (referred to as a scrambler in this manual) and prescrambler are used to convert X and Y addresses generated from the ALPG and to apply them to the DUT.

The scrambler has scramble memory and is used to convert addresses according to the scramble memory data.

The prescrambler has prescramble memory and is used to convert addresses according to the prescramble memory data.

The scramble program is used to create scramble data. The scramble program can be used for both the scrambler and prescrambler. (Some functions excluded.)



The information in this manual refers to the scramble program and also applies to the prescrambler unless otherwise specified.

The input and output addresses for the scrambler and prescrambler are expressed as shown below.

Input: Addresses generated from the ALPG

Output: Addresses applied to the DUT

These expression may change, depending on how to use the scrambler or prescrambler (For example, if the prescrambler is used, addresses are transferred from the prescrambler to the scrambler. If the prescrambler is not used, addresses are transferred from the ALPG to the scrambler.)

➔ For more information on the scrambler and prescrambler, refer to • ["Scrambler and Prescrambler Block Diagram" on page Preface-2.](#)

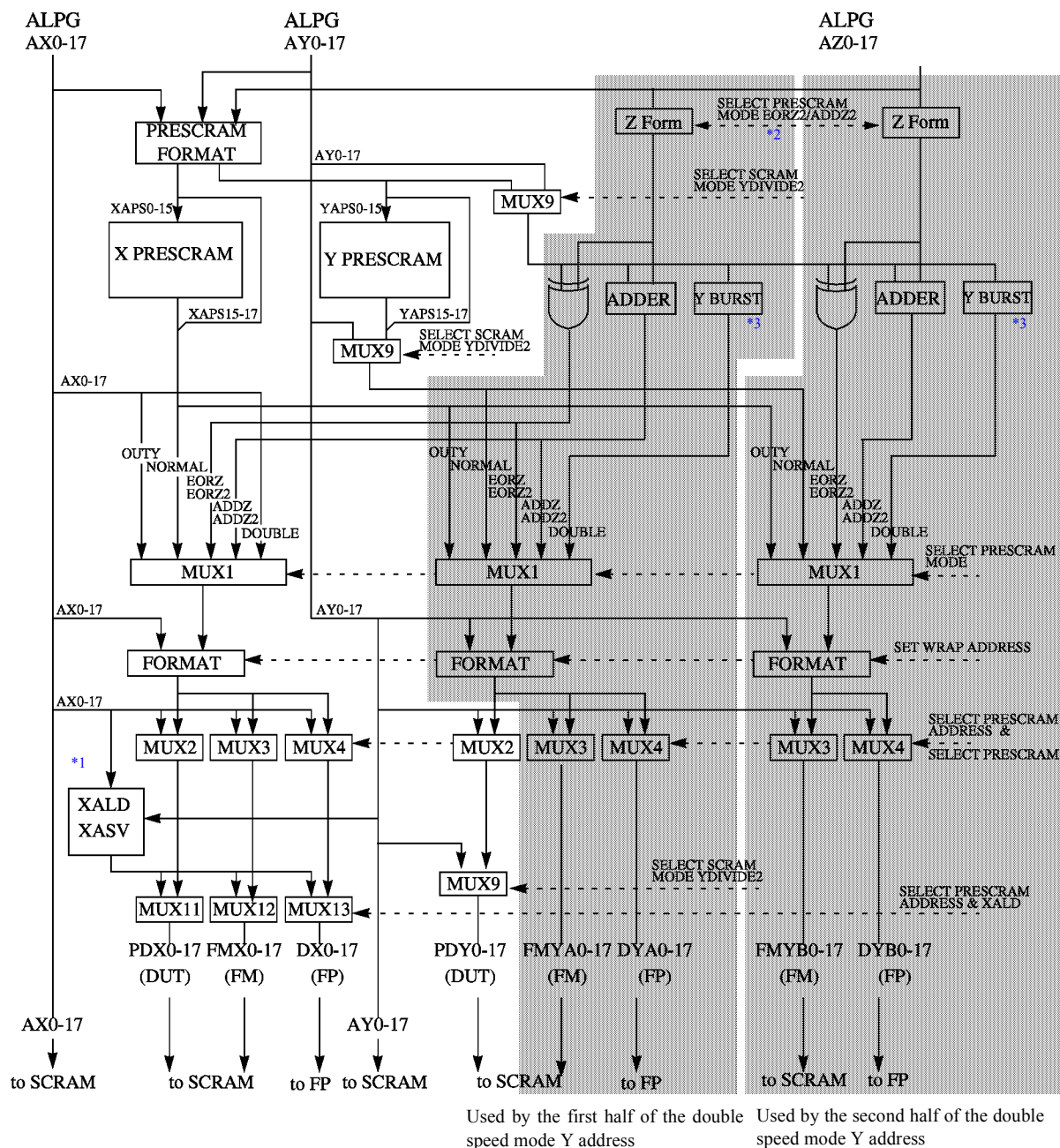
- Compatibility

ATL-51 source scramble programs are compatible with T5300 series (MOST-50 and ASX/U-50) source scramble programs. However, T5300 series object scramble programs cannot be executed.

These source programs can be compiled and executed.

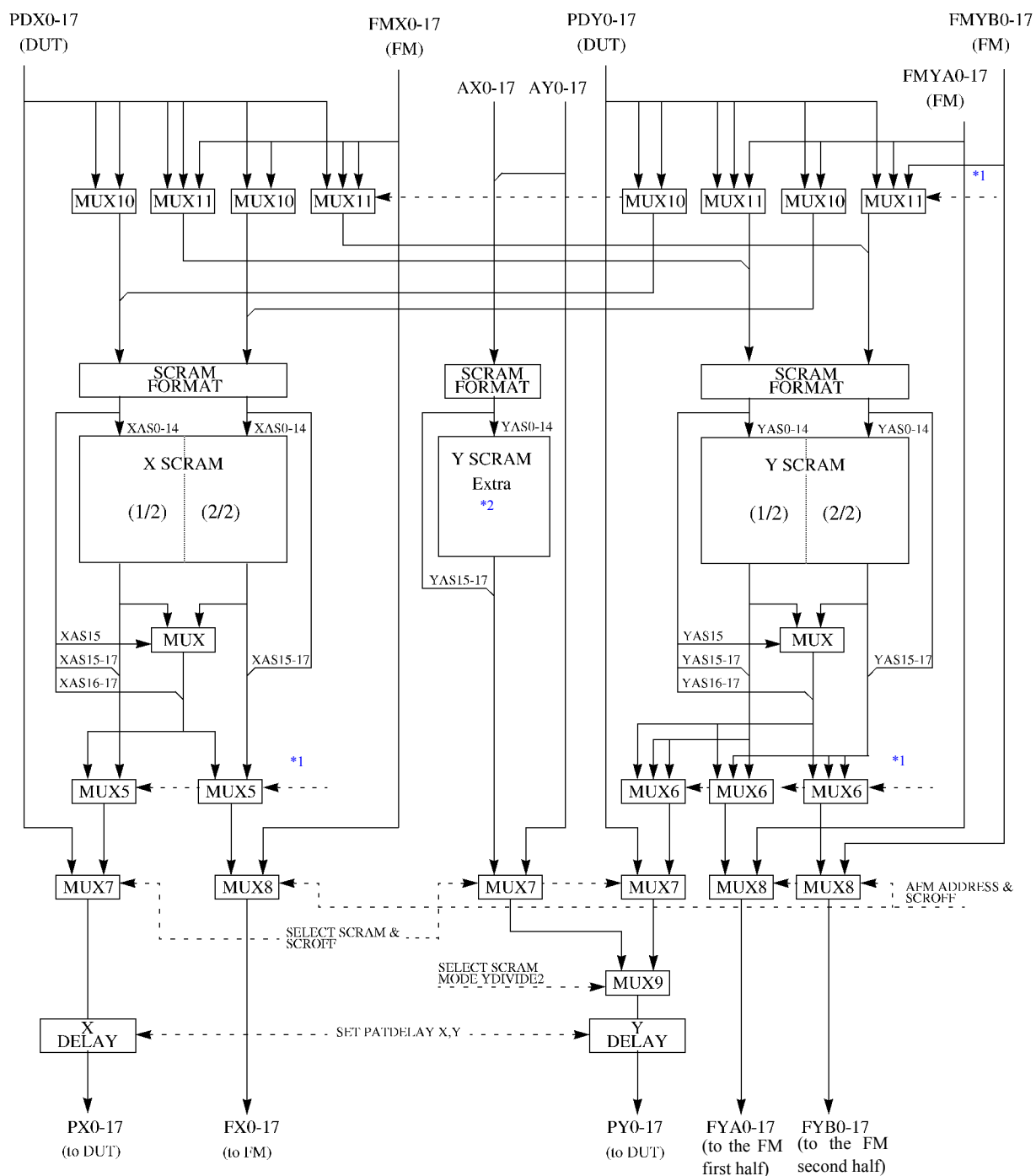
- Scrambler and Prescrambler Block Diagram

T5593



*1 Saves the X address of the step where the pattern program XASV instruction is specified.
The saved X address is loaded by using the XALD instruction.

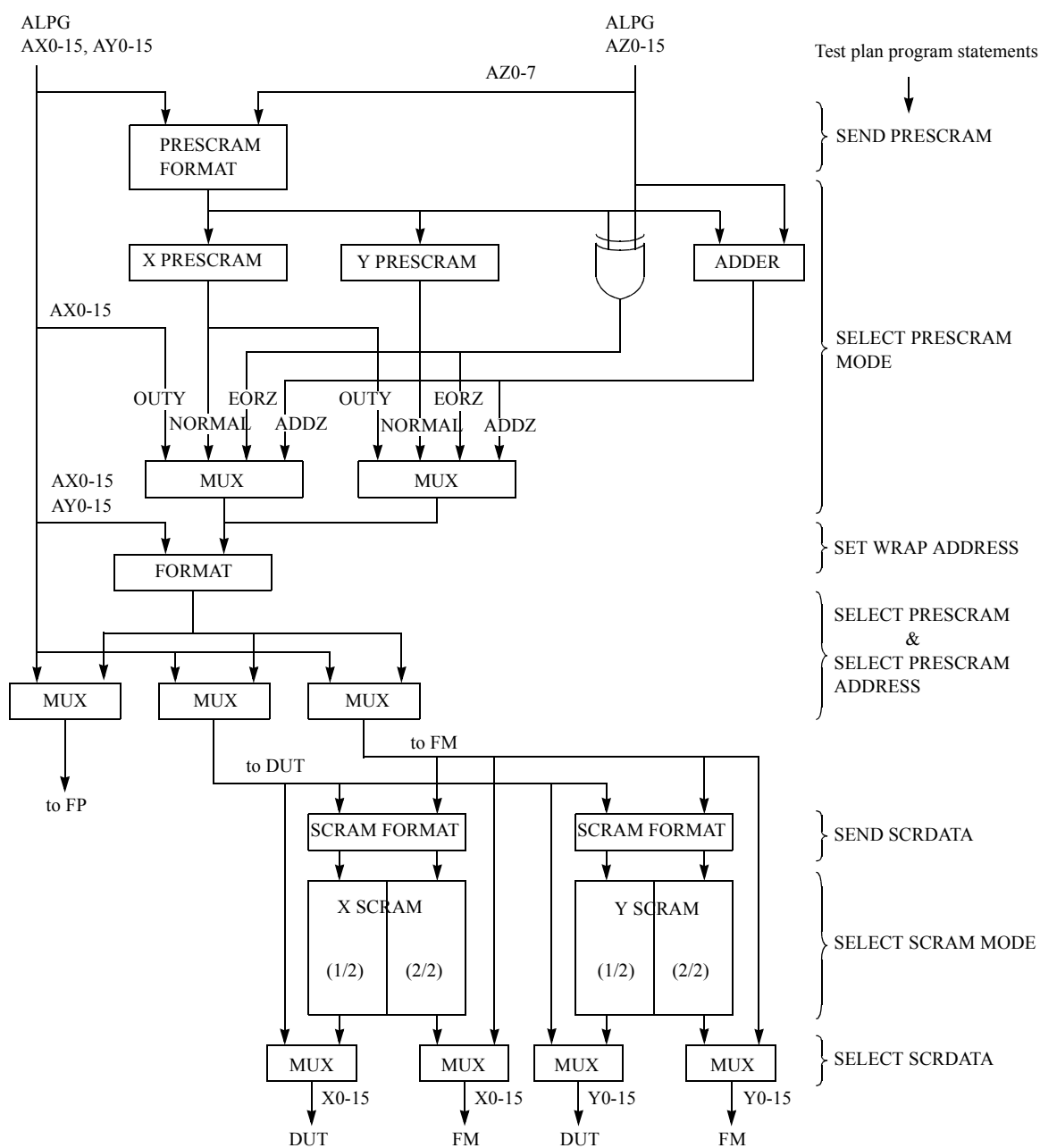
- *2 When EORZ2 or ADDZ2 is specified by using the SELECT PRESCRAM MODE statement, Z Form generates different addresses to each EOR or ADDER circuit used for the first half of Y address and the second half of Y address in the double speed mode. (For the first half, an address is generated, where z address is shifted by one bit to the left and the LSB is set to FL(0). For the second half, an address is generated, where z address is shifted by one bit to the left and the LSB is set to FH(1).)*
- *3 When YBURST is specified by using the SELECT PRESCRAM MODE statement, Y BURST generates different Y addresses to the first and the second halves of the Y address in the double speed mode. (For the first half, Y address is generated. For the second half, the Y address where the LSB is set to FH(1) is generated.)*



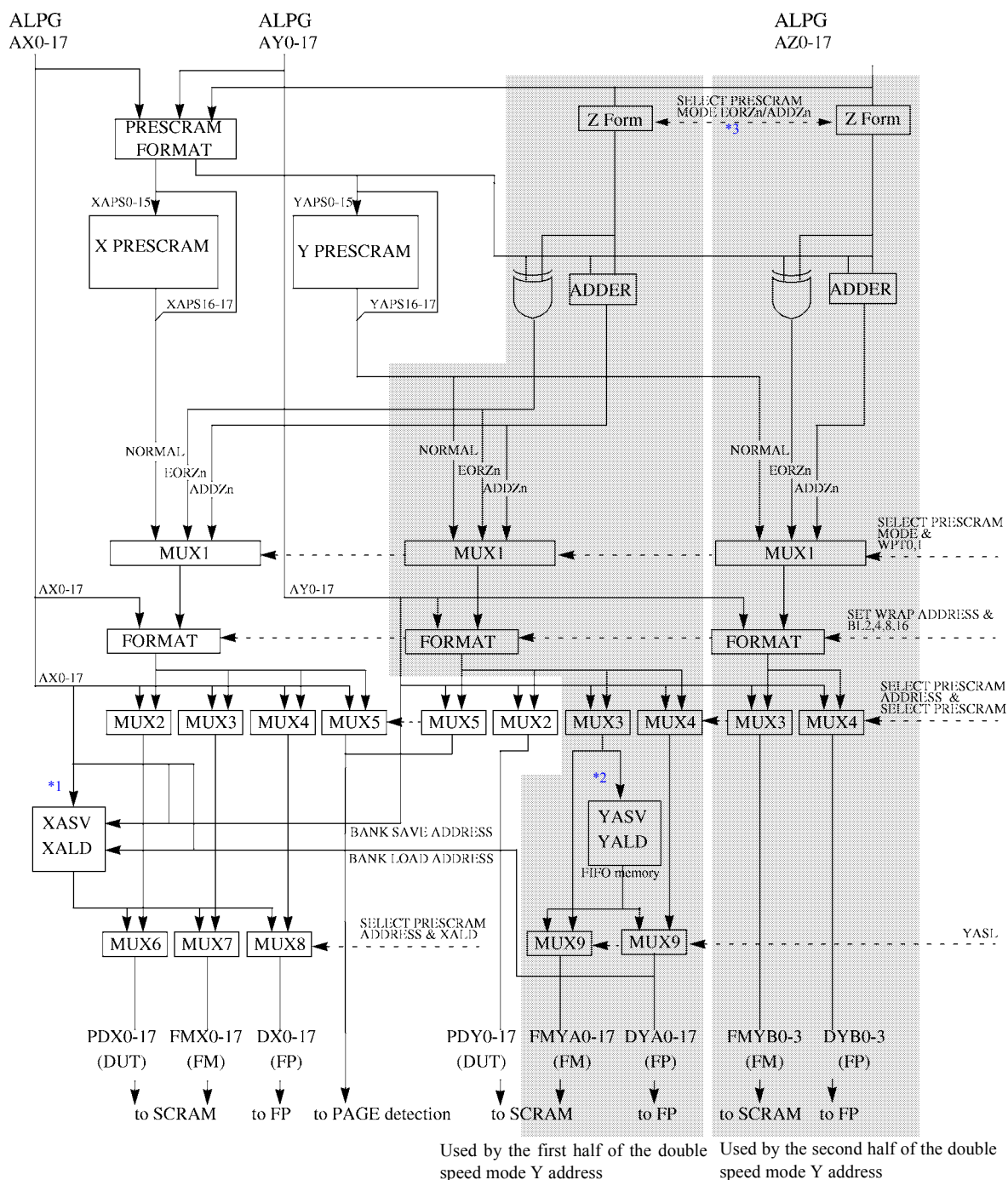
*1 MUX5, 6, 10, and 11 select addresses depending on the mode specified by the SELECT SCRAM MODE statement.

*2 When YDIVIDE2 is specified by the SELECT SCRAM MODE, Y PRESCRAM is used to convert the address on the Y SCRAM DUT side.

T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P



T5377

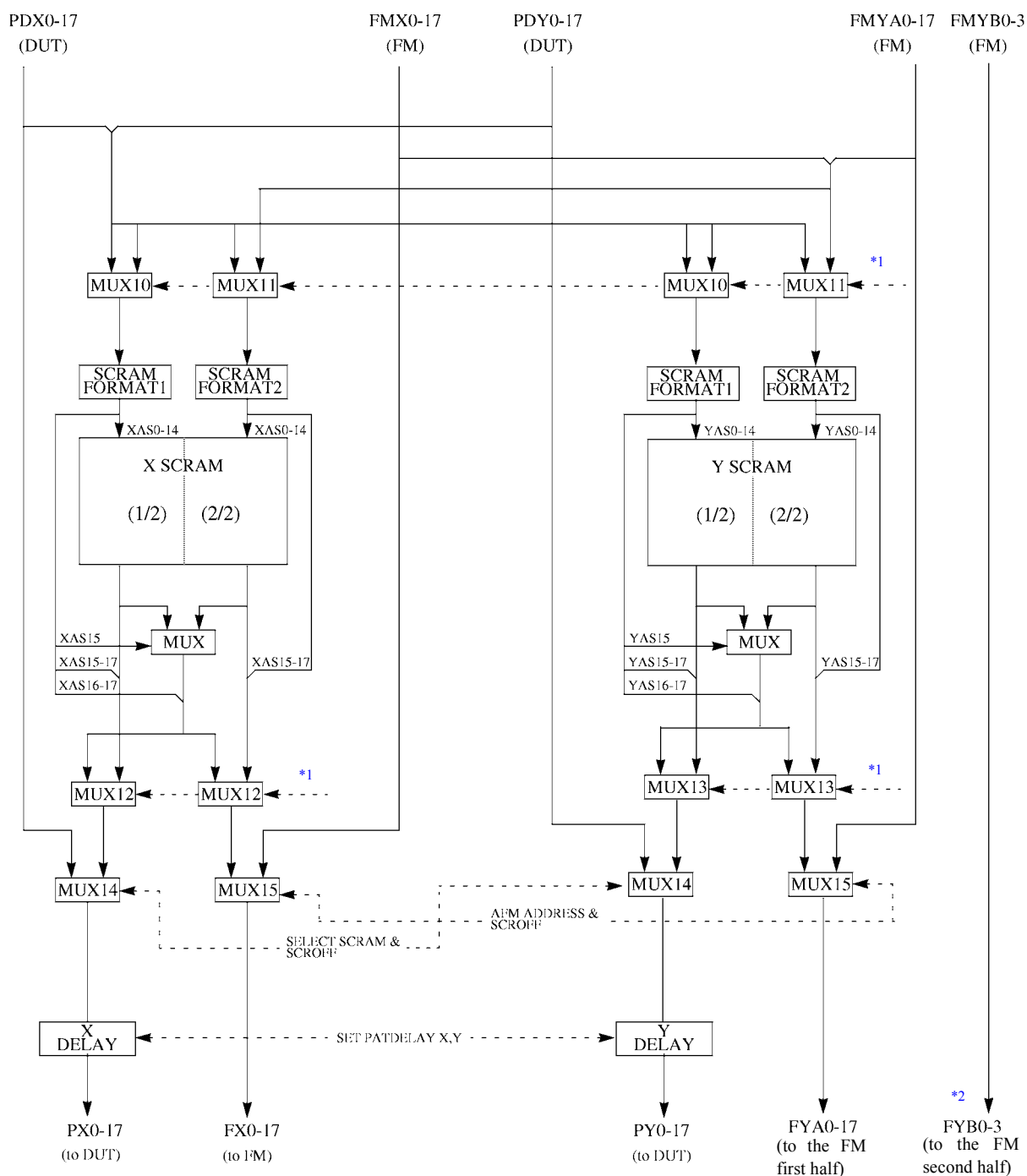


*1 Saves the X address of the step where the pattern program XASV instruction is specified.
The saved X address is loaded by using the XALD instruction.

**2 Saves the Y addresses of the step, where the pattern program YASV instruction is specified, to the FIFO memory.*

The saved Y addresses move the FIFO memory pointer by using the YALD instruction and load the step where the YASL instruction is specified.

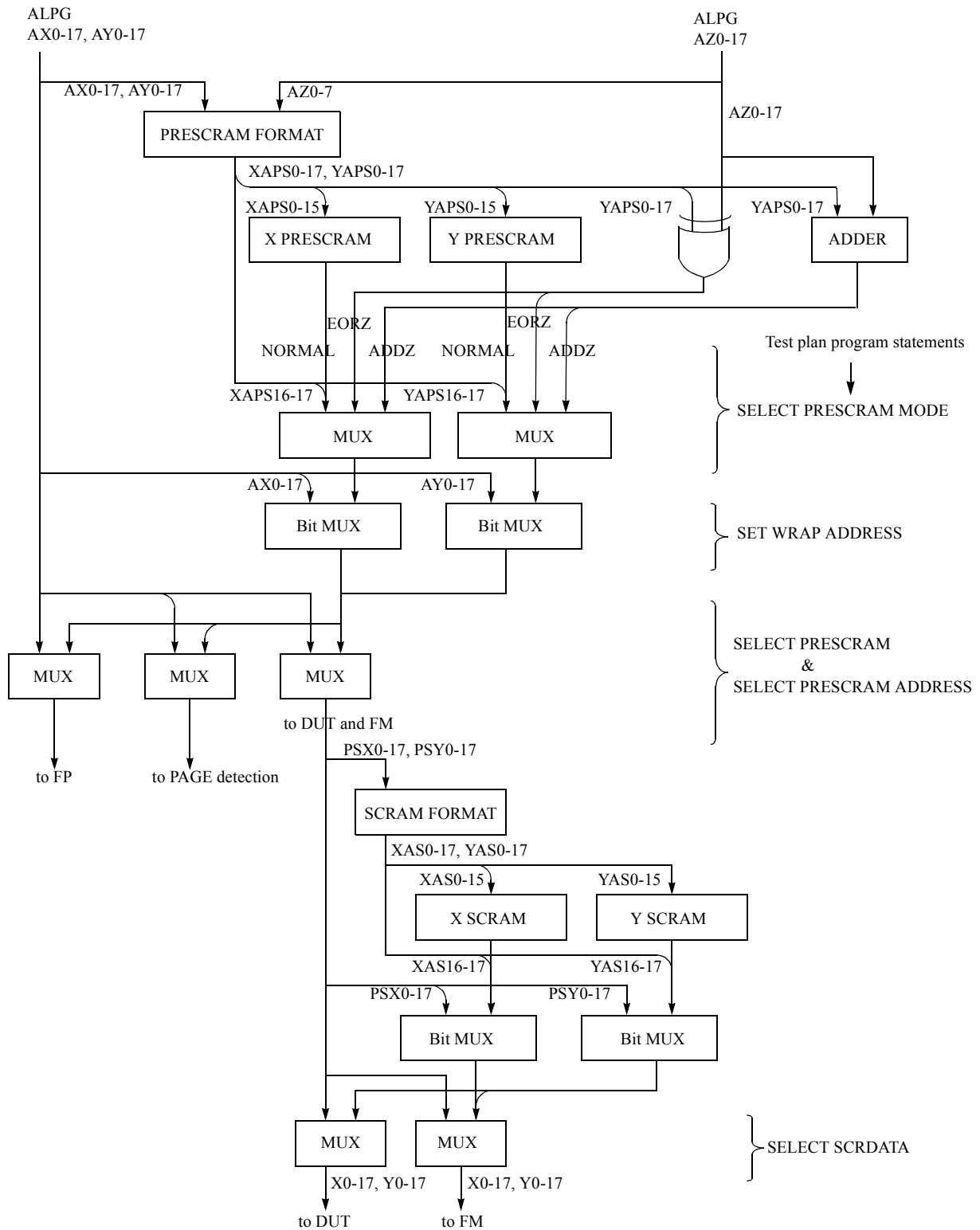
**3 When EORZ2 to 4/ADDZ2 to 4 are specified by using the SELECT PRESCRAM MODE statement, Z Form generates different Z addresses to each EOR or ADDER circuit which is used for the first half and the second half of the Y addresses in the double speed mode.*



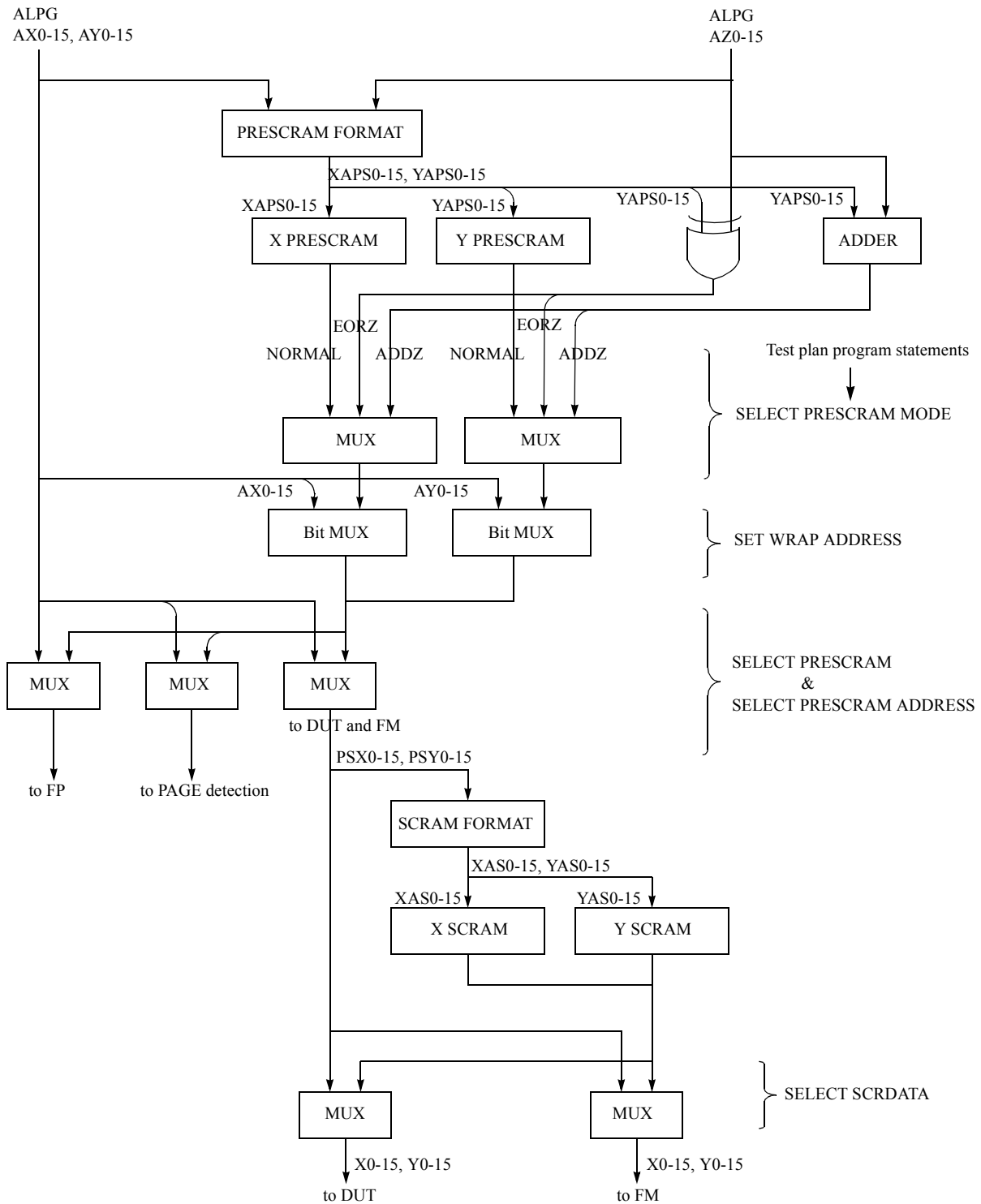
*1 MUX10 to 13 select addresses depending on the mode specified by the SELECT SCRAM MODE statement.

*2 The 14 high-order bits of the second half of the Y addresses in the FM are the same as those in the first half of the Y addresses in the FM.

T5376 and T5375



T5371



- Scrambler and prescrambler configuration

Test system	X and Y Addresses	X Scramble Memory	Y Scramble Memory	X Prescramble Memory	Y Prescramble Memory
T5593	18 bits	64K × 16 bits	64K × 16 bits	32K × 15 bits	32K × 15 bits
T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P	16 bits	64K × 16 bits	64K × 16 bits	32K × 15 bits	32K × 15 bits
T5377, T5376 and T5375	18 bits	64K × 16 bits	64K × 16 bits	64K × 16 bits	64K × 16 bits
T5371	16 bits	64K × 16 bits	64K × 16 bits	64K × 16 bits	64K × 16 bits
T5336 and T5311	16 bits	64K × 16 bits	64K × 16 bits	None	None

Related Manuals

ATL-51 Test Plan Program Reference Manual(No. : 8431498)

ATL-51 Pattern Program Reference Manual(No. : 8256292)

ATL-51 Pattern Program Reference Manual (T5377/T5376/T5375/T5371/T5336/T5311)(No. : 8289058)

ATL-51 Socket Program Reference Manual(No. : 8256293)

T5500 Series Tester Utility Program Reference Manual(No. : 8256295)

T5500 Series Controller Command Reference Manual(No. : 8256297)

ATLworks Tool User's Manual(No. : 8311174)

ASX/U-51 Hybrid Controller User's Manual(No. : 8409598)

ASX/U-51 Hybrid Controller Reference Manual(No. : 8409600)

Error Code Table Reference Manual(No. : 8141091)

XATL/U-51 Cross Compiler User's Manual(No. : 8256237)

ADVANsite Test System Software Installation/System Start User's Manual VOL1 Basic OS Installation(No. : 8227728)

ADVANsite Test System Software Installation/System Start User's Manual VOL2 Tester Environment Software Installation(No. : 8227729)

T5500 Series Memory Test System Control Box/trigger Panel User's Manual(No. : 8256298)

T5300 Series Flash Memory Parallel Measurement Function Description(No. : 8239605)

T5500 Series Memory Test System Calibration Program Reference Manual(No. : 8256296)

T5500 Series Memory Test System Precision Calibration Program Reference Manual(No. : 8311137)

T5500 Series Memory Test System Precision Calibration2 Program Reference Manual(No. : 8350393)

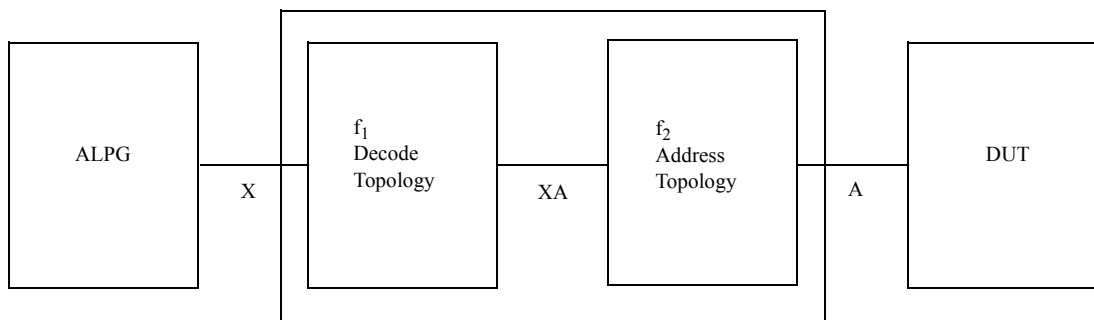
1. General Description

To convert addresses for the decode topology and address topology, a scramble program is required. The scramble program is also useful to specify how to invert data bits at specific addresses.

1. 1 Scramble Program Purpose

The scramble program (SCRAM: Scrambler RAM) is used to convert addresses for memory tests. In many IC memory devices, the physical chip addresses are different from the logical addresses, because they are designed to achieve compatibility with similar products of other companies or to downsize the device. This IC memory structure characteristic is called decode topology and address topology. (See [Figure 1-1](#).)

Figure 1-1 Decode Topology and Address Topology

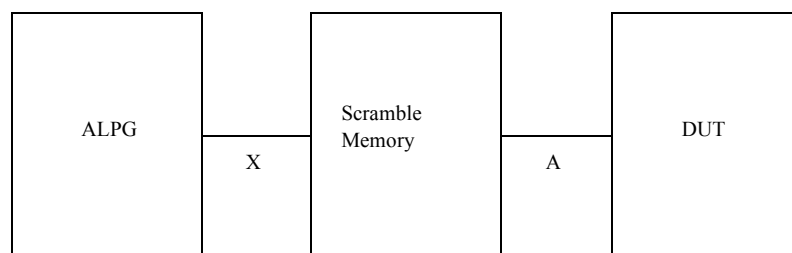


Address A, which is applied to the DUT, can be expressed using functions f_1 and f_2 as shown below.

$$A = f_2 (f_1 (X))$$

When address A is calculated using the above functions and written into scramble memory, the address is converted for testing.

Figure 1-2 Scramble Memory



The scramble program is used to create scramble memory data.

The methods for programming are described below.

- Programming functions f_1 and f_2 in consideration of the decode topology and address topology and generating address A. (See [Figure 1-1.](#))
- The decode and address topology were treated as a black box and address A is generated. (See [Figure 1-2.](#))

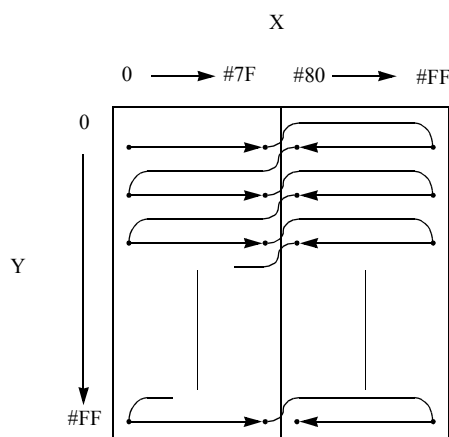
The scramble program is not only used to create scramble memory data but also has the other functions described below.

Creating scramble data for the prescrambler

The scramble program is used to create scramble data for the prescramble memory.

The prescrambler can be used to convert addresses in the same manner as the scrambler. In other words, the prescrambler can convert the ALPG addresses and generate different addresses in sequences that the ALPG is not capable of.

Example: The prescrambler can generate the mirror pattern shown below.



Creating ARIRAM data

ARIRAM is an area inversion function used to invert ALPG pattern data at specified addresses. The scramble program can be used to create ARIRAM data which includes area inversion conditions.

➔ For more information on ARIRAM, refer to [ATL-51 Pattern Program Reference Manual\(No. : 8256292\)](#).

2. *Program Overview*

This chapter briefly describes the scramble program settings and compilation messages.

2. 1 Program Settings

The scramble program begins with the SCRAM compilation control statement and ends with the END statement. Three types of settings described below are available.

- Setting decode and address data (in logical notation)

```
SCRAM name
  RDX radix
  AMAX = x, y
  EXPRESSION

      Decode correction statements

ADDRESS

      Address correction statements

ARIRAM DEFINE

      ARIRAM data setting statements

END
```

name refers to a scramble program name.

x and y refer to the maximum X and Y addresses, respectively.

- Setting addresses and data (individually)

```

SCRAM name
  RDX radix
  AMAX = x, y

      X, Y address/data setting statement

ALINK

      Address bit link statements

ARIRAM DEFINE

      ARIRAM data setting statements

END

```

name refers to a scramble program name.

x and y refer to the maximum X and Y addresses, respectively.

- Combining individual address and data settings with logical equations

```

SCRAM name
  RDX radix
  AMAX = x, y
  REPLACE

      X, Y address/data setting statement

EXPRESSION

      Decode correction statements

ADDRESS

      Address correction statements

ARIRAM DEFINE

      ARIRAM data setting statements

END

```

name refers to a scramble program name.

x and y refer to the maximum X and Y addresses, respectively.

2. 2 Compilation Messages

When the source file is compiled without a syntax error, the messages shown below are displayed.

```
SCRAM f COMPILATION OK!
```

```
SOURCE LINE NUMBERS      : l1  
OBJECT FILE SIZE (BYTE)   : n  
COMPILATION TIME (SEC)    : t  
LINES/MINUTE              : l2
```

```
X MAX ADDRESS             : x  
Y MAX ADDRESS             : y  
Z MAX ADDRESS             : z
```

f

Scramble program name

l₁

Total number of source program lines

n

Object file size (bytes)

t

Compilation time (seconds)

l₂

Number of lines compiled in one minute

x

Maximum X address for scramble data

y

Maximum Y address for scramble data

z

Always 0 (Because the Z scramble data is not generated.)

3. *Handling Characters*

This chapter describes characters and numbers which can be used for the scramble program.

3. 1 **Constants**

Constants beginning with an integer from 0 to 9 are handled according to the radix specified by the RDX statement. If the RDX statement is not specified, they are handled as hexadecimal values.

Constants beginning with # are always handled as hexadecimal numbers regardless of the radix specified by the RDX statement.

The subscripts for X and Y addresses, data, decoder topology, address topology, and ARIRAM data, as well as the address specified by the AMAX statement, are also handled according to the radix specified by the RDX statement.

3. 2 Special Characters

#

Constants beginning with # are always handled as hexadecimal numbers regardless of the radix specified by the RDX statement.

@

This character is used at the end of line to continue the program onto the next line.

4. Common Control Statements

This chapter describes control statements which can be used for the scramble program.

4. 1 Common Control Statements

4. 1. 1 SCRAM

Function: This is a compilation control statement used to indicate the program title.

This statement is used to define a scramble program name.

A program name must be specified by using up to 32 characters which include alphanumeric characters, periods (.), and dollar signs (\$).

Syntax

```
SCRAM program name
```

Example

(1) Naming the Scramble Program EXAMPLE

```
SCRAM EXAMPLE
```

4. 1. 2 RDX

Function: This is a control statement used to define the radix of constants.

Syntax

```
RDX n
```

Argument: n

Refers to a decimal number from 2 to 16.

If omitted, all constants are handled as hexadecimal numbers in the program.

Example

(1) Selecting Decimal Notation

```
RDX 10
```

4. 1. 3 AMAX

Function: This is a control statement used to specify the maximum X and Y addresses of scrambled addresses.

The AMAX statement must be defined before X and Y scramble addresses are specified.

Syntax

```
AMAX = x y          ----- (a)
AMAX = x            ----- (b)
AMAX = Xn1 Yn2      ----- (c)
AMAX = Xn1          ----- (d)
```

Argument: x, y

Refers to the maximum X and Y addresses of the scrambled addresses.

Available range

Test system	Scrambler	Prescrambler
T5500 series	0 to 65535 (#0 to #FFFF)	0 to 32767 (#0 to #7FFF)
T5377, T5376, T5375, and T5371	0 to 65535 (#0 to #FFFF)	0 to 65535 (#0 to #FFFF)
T5336 and T5311	0 to 65535 (#0 to #FFFF)	Unavailable

The numbers in parentheses are expressed in hexadecimal notation.

n₁, n₂

Refers to the maximum bit numbers of the X and Y addresses. A bit number is assigned to each digit of the X and Y addresses, and n₁ and n₂ refer to the maximum bit numbers of the X and Y addresses, respectively.

Available range

Test system	Scrambler	Prescrambler
T5500 series	0 to 15	0 to 14
T5377, T5376, T5375, and T5371	0 to 15	0 to 15
T5336 and T5311	0 to 15	Unavailable

Description

Equations (b) and (d) cannot be used to specify a Y scrambled address.

Specify a maximum X address for the X scrambled addresses and a maximum Y address for the Y scrambled addresses under the following conditions.

- Maximum X address (AMAX) $\geq$ Maximum ALPG X address
- Maximum Y address (AMAX) $\geq$ Maximum ALPG Y address

If the highest values are specified for the maximum X and Y addresses, the above conditions can be omitted.

 Tip

When the DIVIDE or SEPARATE mode is specified for the SELECT SCRAM MODE statement in the test plan program, the scramble memory of 64 Kwords is partitioned into two memory spaces of 32 Kwords each.

Then, specify AMAX within the ranges listed below.

X and Y ranges

0 to 32767(#7FFF)

n₁, n₂ ranges

0 to 14

4. 1. 4 END

Function: This is a compilation control statement used to define the end of a program.

Syntax

```
END
```

4. 1. 5 ARIRAM DEFINE

Function: This is used to declares that the following settings are ARIRAM data.

Syntax

```
ARIRAM DEFINE
```

```
ARIRAM data setting statements
```

Description

This statement can be written in a program only once.

This statement must also be located after the AMAX statement.

Tip

- *ARIRAM DEFINE can also be used for a pattern program. The settings below are listed in the order of descending priority.*

1. *SCRAM specified in a test plan program*
2. *ARIRAM DEFINE specified for the module in a pattern program*
3. *SCRAM specified for the module in a pattern program*
4. *ARIRAM DEFINE specified for the common section in a pattern program*
5. *SCRAM specified for the common section in a pattern program*

- *A SELECT ARIRAM statement is used to enable or disable the ARIRAM data.*

T5377, T5376, T5375, T5371, T5336, and T5311:

When a SELECT ARIRAM statement is not specified or a SELECT ARIRAM NORMAL statement is specified, the MR1 register can be used to enable or disable the ARIRAM data.

- *When the ARIRAM DEFINE statement is specified in either the pattern program or the scramble program, the addresses described below are used for area inversion.*

T5500 series, T5377, T5376, T5375, and T5371:

The addresses specified for the SELECT PRESCRAM ADDRESS statement in the test plan program.

T5336 and T5311:

The addresses generated by the ALPG. (These addresses are not converted by the scrambler.)

- *In the T5593 and T5377, when the double speed mode is specified by the test plan program SELECT PRESCRAM MODE statement, first and second half Y addresses of the double speed mode are generated.*

Because the two sets of ARIRAM are mounted, corresponding to the above two Y addresses, each Y address for the first and second half of the double speed mode can be used as an inversion condition for the TP1 or TP2 data in the ARIRAM.

Either the first half or the second half of the addresses in the double speed mode, which can be used as the inversion conditions, are as follows:

- T5593 : The Y address which follows the specification of the MODE DBFORM statement in a pattern program.
- T5377 : The first half of the Y address in the double speed mode which is used as the TP1 data inversion conditions.
- The second half of the Y address in the double speed mode which is used as the TP2 data inversion conditions.

- *When transferring ARIRAM data in the SEPARATE mode, the ARIRAM data, which is specified in the scramble program for DUTs, is transferred. The ARIRAM data, which is specified in the scramble program for the FM, is ignored.*

4. 1. 6 ARIRAM Data Setting Statements

Function: This is used to invert data bits under the specified conditions.

Syntax

(1) Using INV to Invert the Area

$$X = Xk_1 \ O_1 \ Xk_2 \quad \text{----- (a)}$$

$$Y = Yl_1 \ O_2 \ Yl_2 \quad \text{----- (b)}$$

$$INV = X \ O_3 \ Y \quad \text{----- (c)}$$

For ARIRAM data, equations (a) and (b) are used to set the address range and equation (c) is used to set conditions for inverting data bits.



Tip

- *If the maximum X or Y address defined by AMAX exceeds #FFF, INV cannot be specified.*
- *The INV statement cannot be specified together with the inversion settings of Dn = conditional expression or ARI statements.*

(2) Inverting Each Data Bit (Dn = conditional expression)

$$Dn = Xh_1 \ O_1 \ Yi_{1...} \quad \text{----- (a)}$$

$$Dn,m = Xh_2 \ O_1 \ Yi_{2...} \quad \text{----- (b)}$$

$$Dn-m = Xh_3 \ O_1 \ Yi_{3...} \quad \text{----- (c)}$$

Equation (b) can be specified in combination with equation (c).



Tip

- Up to 12 types of *X* and *Y* address bits can be written on the right side of the equation.

◆ **Example:**

```
D0,1= X1.XOR.(Y0.AND..NOT.X7)
D2   = X1.XOR.X7.XOR.(Y0.AND.X6)
D3   = X1.XOR.X7.XOR.(Y0.AND.X6)
```

*The above examples shows that four types of *X* and *Y* address bits are written on the right side.*

- *These settings cannot be specified together with the inversion settings of INV or ARI statements.*
- *When a conditional expression is defined twice or more for the same data bit, the most recent condition is applied.*

(3) Inverting Each Data Bit (with ARI statements)

```
ARI1 = Xh1 O1 Yi1...
Dn   = ARI1          ----- (a)
Dn,m = ARI1          ----- (b)
Dn-m = ARI1          ----- (c)
```

Equation (b) can be specified in combination with equation (c).



Tip

- Up to 12 types of *X* and *Y* address bits can be written on the right side of the ARI statement.
- These settings cannot be specified together with the inversion setting of an INV statement or *Dn* = conditional expression.
- When ARI1 is defined twice or more for the same data bit, the most recent condition is applied.

Argument: K,l

Refers to subscripts for *X* and *Y* address bits.

The available ranges are listed below. The address range is determined by the maximum address specified by the AMAX statement.

$$0 \leq k \leq 11$$

$$0 \leq l \leq 11$$

O_1, O_2, O_3

Refers to logical operators. The symbols listed below can be used as operators.

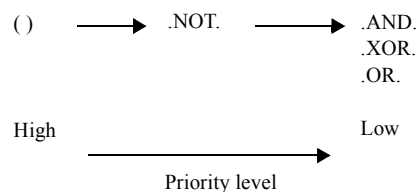
.AND.

.XOR.

.OR.

.NOT.

In addition to the above, parentheses "(")" can also be used. The operational priorities are shown below.



Note that operational expressions with the same priority are performed from the left expression.

Up to 50 operators can also be used.

Up to 127 pairs of parentheses "(")" can also be used.

h, i

Refers to subscripts for X and Y address bits. The available range is shown below.

The test system is the T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, T5571P, T5371, T5336, and T5311.

$$0 \leq h, i \leq 15$$

The test system is the T5593, T5377, T5376, and T5375.

$$0 \leq h, i \leq 17$$

n, m

Refers to data bit numbers. The available range is shown below.

$$0 \leq n, m \leq 17$$

Tip

When the TP 36-bit mode (TPM36) is specified in the pattern program for the T5500 series, T5377, T5376, or T5375, DA18 to DA35 are also inverted under the same conditions as D0 to D17. The conditions for D0 to D17 correspond to DA18 to DA35, respectively. When TP2INV is specified in the pattern program, DB0 through DB35 are also inverted as well as DA0 through DA35.

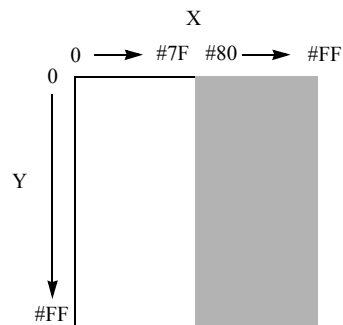
ARII

Specifies data inversion conditions. The available range is shown below.

$$0 \leq l \leq 7$$

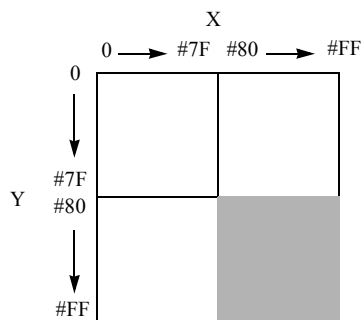
Example

(1) Inversion Applied Only when X7 = 1



```
ARIRAM DEFINE
      X      = X7
      INV     = X
```

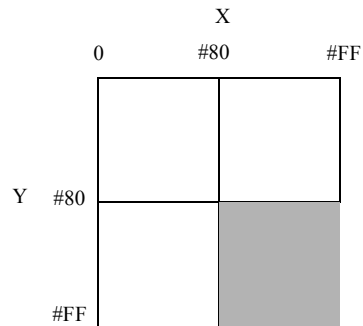
(2) Inversion Applied Only when X7 = 1 and Y7 = 1



```
ARIRAM DEFINE
      X      = X7
      Y      = Y7
      INV     = X.AND.Y
```

The above setting refers to an AND between X7 = 1 and Y7 = 1.

(3) Inverting Data in the Range Shown Below



- Using INV to invert the area

```
AMAX = #FF, #FF
```

```
ARIRAM DEFINE
```

```
    X = X7
```

```
    Y = Y7
```

```
    INV = X.AND.Y
```

- Inverting each data bit

```
AMAX = #FF, #FF
```

```
ARIRAM DEFINE
```

```
    D0-17 = X7.AND.Y7
```


5. *Exclusive Control Statements*

This chapter described how to express the decode topology and the address topology in the scramble program.

5. 1 Setting Decode and Address Topology (In Logical Notation)

5. 1. 1 EXPRESSION

Function: This is used to declare that decode correction statements are listed below.

Syntax

EXPRESSION

Decode correction statements

Description

This statement can be written in a program only once.

Write this statement after the AMAX statement. However, if a REPLACE statement is used, write the statement between the REPLACE and ADDRESS statements.

This statement cannot be written together with an ALINK statement or an X, Y address/data setting statement in the same program. However, if the REPLACE statement is used, this statement can be written together with the X, Y address/data setting statement.

Note that when only EXPRESSION is written, decode topology is not made.

5. 1. 2 Decode Topology (Decode Correction Statements)

The decode correction statement is used to correct the address conversion using logical expressions. Address bits can be selected and applied to the DUT by directly writing XAT15 through XAT17, YAT15 through YAT17 on the left side of the equation.

When XA (XAT) is written on the left side of the equation and Y and Z are written on the right side, Y and Z addresses can be assigned to XA (XAT). When YA (YAT) is written on the left side and X and Z are written on the right side, X and Z addresses can be assigned to YA (YAT).

If OFF is specified on the right side of the decode correction statement, address bits generated by the ALPG are not converted with decode or address correction statements but are applied to the DUT.

```
XAh = Xj O1 Xk O2 Yl O3 Ym O4 Zn O5 Zp
YAh = Xj O1 Xk O2 Yl O3 Ym O4 Zn O5 Zp
XAh = OFF
YAh = OFF
XATi = Xq
XATi = Yr
XATi = Zs
YATi = Xq
YATi = Yr
YATi = Zs
```

Be sure to write EXPRESSION before the above settings.

XAh, YAh

Refers to logical operation results. These are used as input data for the address topology. The subscripts depend on the AMAX statement setting.

If OFF is specified on the right side of the equation, address bits generated by the ALPG are directly applied to the DUT. (They are not used as input data for the address topology.)

When the RDX statement is not written, all the specified values are treated as hexadecimal numbers. Therefore, specify RDX 10 when XA10 to XA15 and YA10 to YA15 are written.

XATi, YATi

These are address bits applied to DUTs. Address bits written on the right side are not used for the address topology but are applied to the DUT. Since the address bits written on the right side are applied directly to DUTs, logical operations are not performed for them. The Subscript range is shown in [Table 5-1](#).

Table 5-1 Subscript Range

Test system	Scrambler	Prescrambler
T5593	16 ≤ XATi, YATi ≤ 17	15 ≤ XATi, YATi ≤ 17
T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P	Unavailable	XAT15 and YAT15
T5377, T5376 and T5375	16 ≤ XATi, YATi ≤ 17	16 ≤ XATi, YATi ≤ 17
T5371, T5336, and T5311	Unavailable	Unavailable

 **Tip**

When the number of right side elements in the X decode correction statement is 16, XAT15 cannot be specified.

5. 1 Setting Decode and Address Topology (In Logical Notation)

When the number of right side elements in the Y decode correction statement is 16, YAT15 cannot be specified.

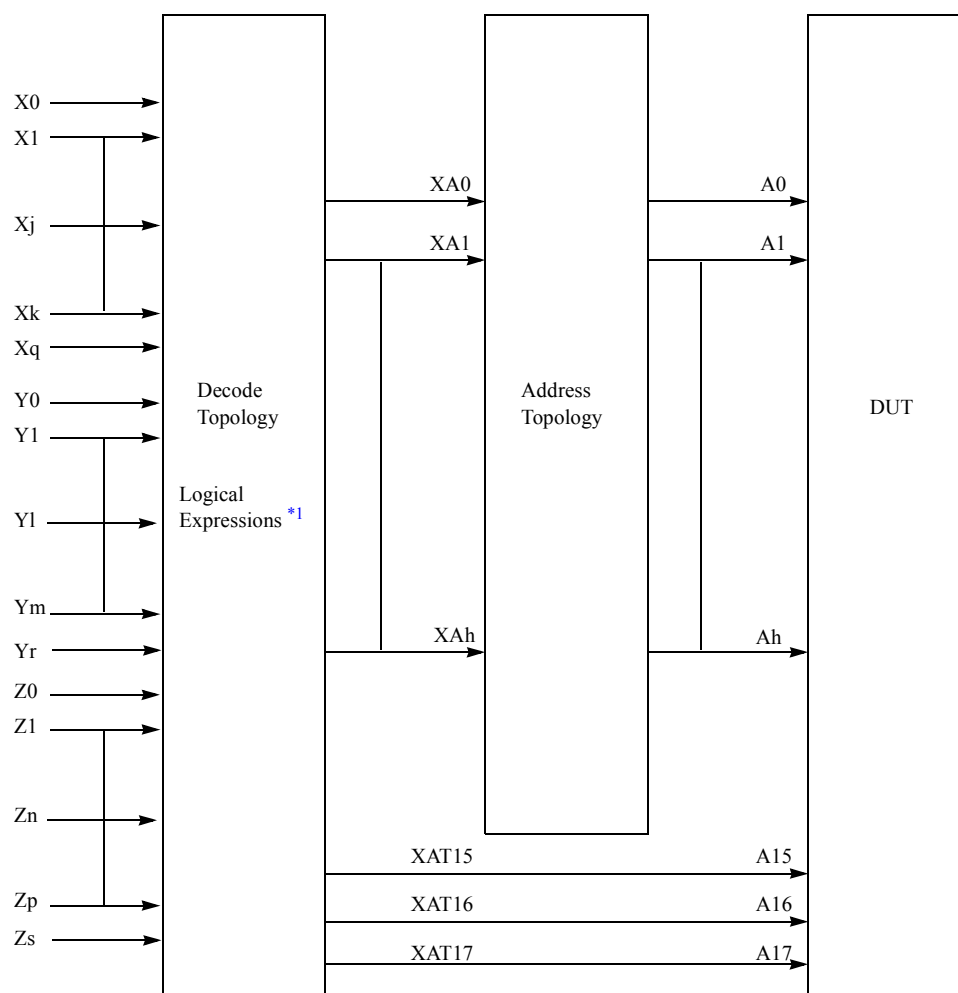
When the RDX statement is not written, all the specified values are treated as hexadecimal numbers. Therefore, specify RDX 10 when XAT15 to XAT17 and YAT15 to YAT17 are written.

Xj, Xk, Yl, Ym, Zn, Zp, Xq, Yr, Zs

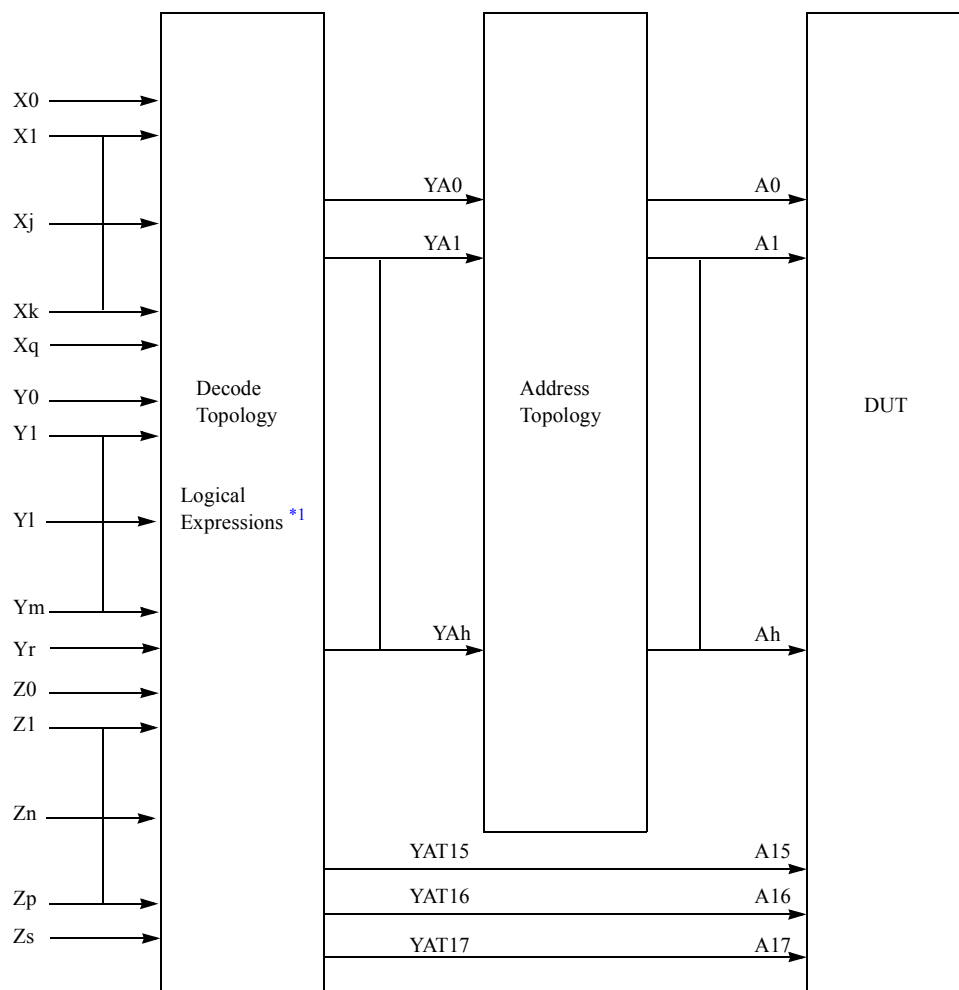
Refers to address bits generated by the ALPG. Each bit size is 1.

The available subscripts depend on the ALPG address range of each test system shown below.

Figure 5-1 X Address Decode Topology



*1 The logical operation results are not output to XAT15 through XAT17.

Figure 5-2 Y Address Decode Topology

*1 The logical operation results are not output to YAT15 through YAT17.

Table 5-2 Subscript Range

Test system	Scrambler	Prescrambler
T5593	$0 \leq X, Y \leq 17$	$0 \leq X, Y \leq 17$ $0 \leq Z \leq 7$
T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P	$0 \leq X, Y \leq 15$	$0 \leq X, Y \leq 14$ $0 \leq Z \leq 7$
T5377, T5376 and T5375	$0 \leq X, Y \leq 17$	$0 \leq X, Y \leq 17$ $0 \leq Z \leq 7$
T5371	$0 \leq X, Y \leq 15$	$0 \leq X, Y \leq 15$ $0 \leq Z \leq 7$

5. 1 Setting Decode and Address Topology (In Logical Notation)

Test system	Scrambler	Prescrambler
T5336 and T5311	$0 \leq X, Y \leq 15$	Unavailable

When address bits are not specified for decode correction statements, the same address bits (XAh, YAh, XATi, and YATi) are allocated to both the right and left sides by default. (The default allocation is performed within the maximum address range of the X and Y scramble memory data defined in the AMAX statement.)

The address bits written on the right side and the default address bits are used for decode correction.

◆ **Default Setting Example**

```
SCRAM SAM1
RDX 10
AMAX=###F, #1F
EXPRESSION
  XA0 =X0.XOR.Y0
  XA1 =X1.XOR.Y1
  XA2 =X2.XOR.Y2
  XA3 =X3.XOR.Y3
```

```
YA0 =Y0.XOR.X1
```

```
END
```

```
SCRAM SAM2
RDX 10
AMAX=###F, #1F
EXPRESSION
  XA0 =X0.XOR.Y0
  XA1 =X1.XOR.Y1
  XA2 =X2.XOR.Y2
  XA3 =X3.XOR.Y3
  XA4 =X4
  XA5 =X5
  XA6 =X6
  XA7 =X7
  XA8 =X8
  XA9 =X9
  XA10=X10
  XA11=X11
```

```
YA0 =Y0.XOR.X1
YA1 =Y1
YA2 =Y2
YA3 =Y3
YA4 =Y4
```

```
END
```

In the SAM1 program, XA4 to XA11 and YA1 to YA4 are omitted. The SAM1 and SAM2 programs are the same because the defaults are applied to the omitted address bits.

Right side elements

SAM1 and SAM2

- X0 to X11 and Y0 to Y3 for X decode correction
- Y0 to Y4 and X1 for Y decode correction

OFF

The address bits set to OFF are excluded from the address conversion. As a result, the ALPG address bits are not converted but directly applied to the DUT.

OFF can be specified for XAh and YAh in the T5593, T5377, T5376, and T5375 test systems.

To effectively use the decode correction elements, specify OFF for the address bits which are not converted.

◆ Example of OFF Settings and Elements

```
SCRAM SAM2
RDX 10
AMAX=#FFF,#1F
EXPRESSION
  XA0 =X0.XOR.Y0
  XA1 =X1.XOR.Y1
  XA2 =X2.XOR.Y2
  XA3 =X3.XOR.Y3
  XA4 =X4
  XA5 =X5
  XA6 =X6
  XA7 =X7
  XA8 =X8
  XA9 =X9
  XA10=X10
  XA11=X11

  YA0 =Y0.XOR.X1
  YA1 =Y1
  YA2 =Y2
  YA3 =Y3
  YA4 =Y4
END
```

```
SCRAM SAM3
RDX 10
AMAX=#FFF,#1F
EXPRESSION
  XA0 =X0.XOR.Y0
  XA1 =X1.XOR.Y1
  XA2 =X2.XOR.Y2
  XA3 =X3.XOR.Y3
  XA4 =OFF
  XA5 =OFF
  XA6 =OFF
  XA7 =OFF
  XA8 =OFF
  XA9 =OFF
  XA10=OFF
  XA11=OFF

  YA0 =Y0.XOR.X1
  YA1 =OFF
  YA2 =OFF
  YA3 =OFF
  YA4 =OFF
END
```

In the SAM3 program, OFF is specified for address bits which are not converted in the SAM2 program. Address bits set to OFF are not converted. As a result, the same addresses are applied to both the SAM2 and SAM3 programs.

Right side elements

SAM2

- X0 to X11 and Y0 to Y3 for X decode correction
- Y0 to Y4 and X1 for Y decode correction

SAM3

- X0 to X3 and Y0 to Y3 for X decode correction
- Y0 and X1 for Y decode correction

5. 1 Setting Decode and Address Topology (In Logical Notation)

For the SAM2 decode correction, 16 elements are used because X4 to X11 (which are not converted) are treated as elements. Eight elements are used for the X decode correction in the SAM3 program because X4 to X11 are set to OFF and excluded from the decode correction.

Up to 16 (or 15) elements can be specified for the X and Y decode correction as described in the tip section below. As a result, no additional elements can be added to the X decode correction in the SAM2 program. However, up to eight elements can be added to the X decode correction in the SAM3 program.

 Tip

1. Up to 16 elements can be written on the right side of each X and Y decode correction statement. However, the number of elements is limited to 15 under the conditions described below.

- The prescrambler is used for the T5500 series.
- The DIVIDE or SEPARATE mode is specified for the SELECT SCRAM MODE statement in the test plan program.

The number of elements, including default elements, used for the right side are counted.

Elements written on the right sides of the XATi or YATi equation are not counted.

2. OFF cannot be specified for the right side of the decode correction statement under the conditions described below.

- The prescrambler is used.
- A REPLACE statement is used
- The test system is the T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, T5571P, T5371, T5336, or T5311.

3. When transferring the scramble program, in which OFF is specified on the right side of the decode correction statement, the following settings for DUTs and the FM must be the same in the SEPARATE mode:

- Address bits for which OFF is specified.
 - AMAX statement
-

O₁, O₂, O₃, O₄, and O₅

Refers to logical operators. The symbols listed below can be used as operators.

.NOT.

.AND.

.NAND.

.OR.

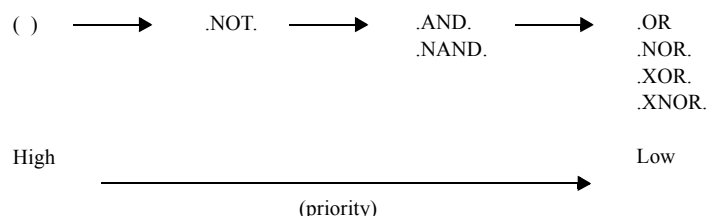
.NOR.

.XOR.

.XNOR.

5. 1 Setting Decode and Address Topology (In Logical Notation)

In addition to the above, parentheses "(")" can also be used. The operational priorities are shown below.



Note that operational expressions with the same priority are performed from the left expression.

Up to 50 operators can be used.

Up to 127 pairs of parentheses "(" ")" can also be used.

◆ About Assignment

An assignment is to write Y or Z in the right side of the X decode correction statement, or to write X or Z in the right side of the Y decode correction statement. The bit used for an assignment is called an assignment bit. (If a REPLACE statement is written, the assignment is disabled.)

"Assignment Example" on page 5-8, Y0 is required to determine the X address in the SAM4 program. Therefore, Y0 is assigned to the X bit area to generate the target address. The assignment bits are allocated in the following order.

X address

X10, X11, X9, X8, ... ,X1, X0, X12, X13, X14, X15

Y address

Y10, Y11, Y9, Y8, ... ,Y1, Y0, Y12, Y13, Y14, Y15

◆ Assignment Example

```

SCRAM    SAM4
RDX      10
AMAX=#F, #1F
EXPRESSION
    XA0=X0.XOR.Y0
    XA1=X1
    XA2=X2
    XA3=X3

    YA0=Y0.XOR.X1
    YA1=Y1
    YA2=Y2
    YA3=Y3
    YA4=Y4
END

```

Since the maximum X address is set to #F with the AMAX statement and X10 is not used as an element in the SAM4 program, Y0 is assigned to X10.

If the maximum X address is #7FF (11 bits) in the SAM4 program, Y0 is assigned to X11 according to the assignment order because X10 is used as an element.

5. 1. 3 ADDRESS

Function: This is used to declare that address correction statements are listed below.

Syntax

```
ADDRESS
```

```
    Address correction statements
```

Description

This statement can be written in a program only once.

This statement must also be located after the AMAX statement. In addition, if the REPLACE or EXPRESSION statement is used, this statement should be specified after the REPLACE or EXPRESSION statement.

This statement cannot be written together with an ALINK statement or an X, Y address/data setting statement in the same program. However, if the REPLACE statement is used, this statement can be written together with the X, Y address/data setting statement.

When only an ADDRESS statement is written, address topology is not made.

5. 1. 4 Address Topology (Address Correction Statements)

The address correction statement is used to convert addresses according to the address topology.

◆ Individual Setting

```
Xi = XAj  
Yi = YAj
```

◆ Range Setting

```
Xm0 - mi = XAn0 - nj  
Ym0 - mi = YAn0 - nj
```

◆ List Setting

$$Xm_0, m_1, \dots, m_i = XAn_0, n_1, \dots, n_j$$

$$Ym_0, m_1, \dots, m_i = YAn_0, n_1, \dots, n_j$$
XAj, YAj

Refers to the address bits which are output through the decode topology.

Xi, Yi

Refers to the address bits which are output through the address topology.

These addresses are applied to the DUT.

An ADDRESS statement is required before these settings.

Individual setting

Defines that Xi for the address topology corresponds to XAj for the decode topology.

Range setting

Defines that Xm₀ to Xm_i for the address topology correspond to XAn₀ to XAn_j for the decode topology.

List setting

Defines that Xm₀, Xm₁, ..., and Xm_i for the address topology correspond to XAn₀, XAn₁, ..., and XAn_j for the decode topology, respectively.

The three types of settings described above refer to X address settings. They also apply to Y address settings.

The settings can be combined together under the conditions described below.

- Number of address bits written in the left side is equal to the number of address bits written in the right side.
- The subscript is less than the AMAX statement setting.

Address bits which are not written on the left side are sequentially assigned to address bits which are not written on the right side.

 Tip

1. When performing address topology for the address bits which were set to OFF in the decode correction statements, different address bits cannot be specified for both left and right sides of the address correction statements as shown in the example below.

5. 1 Setting Decode and Address Topology (In Logical Notation)

◆ Example

```

SCRAM  SAM5
RDX  10
AMAX=#F

EXPRESSION
  XA2=OFF

ADDRESS
  X0=XA0.XOR.XA1
  X1=XA1
  X2=XA3 ← Unavailable
  X3=XA2 ← Unavailable
END

```

2. *If the decode correction statements are used to set address bits to OFF, the address correction statements for the address bits cannot be omitted.*

◆ Example

```

SCRAM  SAM6
RDX  10
AMAX=#F

EXPRESSION
  XA3=OFF

ADDRESS
  X0=XA0
  X1=XA0
    ← X2=XA2 cannot be omitted
    ← X3=XA3 cannot be omitted
END

```

5. 2 Setting Addresses and Data (Individually)

This section describes how to specify scramble memory data (scrambled addresses) without using EXPRESSION and ADDRESS statements. In other words, X and Y scrambled addresses are specified individually and stored in the scramble memory.

5. 2. 1 X Address and Y Address/Data Setting Statement

Function: This statement is used to specify scramble memory data for each scramble memory address.

Syntax

(1) Individual Setting

$$\begin{aligned} Xa_0 &= d_0 \\ Ya_0 &= d_0 \end{aligned}$$

(2) Range Setting

$$\begin{aligned} Xa_1 - a_1 &= d_1 - d_1 \\ Ya_1 - a_1 &= d_1 - d_1 \end{aligned}$$

(3) List Setting

$$\begin{aligned} Xa_{l+1} \ a_{l+2} \ \dots \ a_m &= d_{l+1} \ d_{l+2} \ \dots \ d_m \\ Ya_{l+1} \ a_{l+2} \ \dots \ a_m &= d_{l+1} \ d_{l+2} \ \dots \ d_m \end{aligned}$$

Description

Individual setting

The scramble memory data (d_0) is assigned to the scramble memory address (Xa_0, Ya_0).

Range setting

The scramble memory data from d_l to d_l are assigned to the scramble memory addresses (Xa_l to Xa_l, Ya_l to Ya_l).

List setting

The scramble memory data $d_{l+1}, d_{l+2}, \dots, d_m$ is assigned to the scramble memory addresses (Xa_{l+1}, Ya_{l+1}), (Xa_{l+2}, Ya_{l+2}), $\dots$ (Xa_m, Ya_m).

The three expressions above can be combined under the following conditions:

- The number of addresses written on the left side of the equation is equal to the number of scramble memory data settings written on the right side.
- The address and data size is less than the AMAX statement setting.

The default addresses are assigned to the addresses which are not written on the left side as described below.

Not using the REPLACE statement

Unassigned addresses (which are not specified on the right side) are assigned to addresses which are not written on the left side in ascending order.

➔ For more information, refer to [Appendix 1. "Scramble Program Example" on page A1-1.](#)

A REPLACE statement is used

The addresses which are not written on the left side are assigned to the same addresses. (The equations have the same address on the left and right side.)


Tip

- *X, Y address/data setting statements do not require a declaration statement. However, if the X, Y address/data setting statements are used together with EXPRESSION or ADDRESS statement, use REPLACE as the declaration statement. If REPLACE is not used, the X, Y address/data setting statement cannot be used together with the EXPRESSION or ADDRESS statement.*
- *Only REPLACE can be written before the X, Y address/data setting statement.*

5. 2. 2 **ALINK**

Function: This is used to declare that the address bit linkage statements are listed below.

Syntax

ALINK Address bit linkage statements
--

Description

This is a declaration statement used to specify how the address bits generated by the ALPG are linked to scramble memory or DUT address bits.

When the declaration statement is written before address bit linkage statements, the address bit linkage statements can be used to assign Y or Z to X as well as logical equations. (This also applies to Y address settings.)

This statement can also be used together with X, Y address/data setting statements.

This statement cannot be used with EXPRESSION, ADDRESS and REPLACE statements in the same program.

This statement can be written in a program only once.

Do not use ALINK before the AMAX statement and X, Y address/data settings.

5. 2. 3 Address Bit Linkage Statements

These statements are used to specify a link between ALPG address bits and scramble memory or DUT address bits.

◆ Individual Setting

```
Xi = AXj
Xi = AYj
Xi = AZj
Yi = AXj
Yi = AYj
Yi = AZj
```

◆ Range Setting

```
Xm0-mi = AXn0-nj
Xm0-mi = AYn0-nj
Xm0-mi = AZn0-nj
Ym0-mi = AXn0-nj
Ym0-mi = AYn0-nj
Ym0-mi = AZn0-nj
```

◆ List Setting

```
Xm0, m1, ..., mi = AXn0, AXn1, ..., AXnj
Xm0, m1, ..., mi = AYn0, AYn1, ..., AYnj
Xm0, m1, ..., mi = AZn0, AZn1, ..., AZnj
Ym0, m1, ..., mi = AXn0, AXn1, ..., AXnj
Ym0, m1, ..., mi = AYn0, AYn1, ..., AYnj
Ym0, m1, ..., mi = AZn0, AZn1, ..., AZnj
```

◆ X, Y, and Z Settings

```
Xm0, m1, ..., mi = AXnj, AYnj, ..., AZnj
Ym0, m1, ..., mi = AXnj, AYnj, ..., AZnj
```

The ALINK statement is required for the address link settings shown above.

The left side of each equation indicates scramble memory address bits. When the subscript value is 16 or 17 bits, the address bits are applied to the DUT. (In the T5500 series prescrambler, when the subscript value is 15 bits, the address bits are also applied to the DUT.)

The right side of each equation indicates address bits generated by the ALPG.

Individual setting

The address bit X_j (Y_j or Z_j) generated by the ALPG is assigned to the scramble memory address bit X_i .

Range setting

The address bits generated by the ALPG, X_{n_0} (Y_{n_0} or Z_{n_0}) to X_{n_j} (Y_{n_j} or Z_{n_j}) are assigned to the scramble memory address bits from X_{m_0} to X_{m_i} , respectively.

List setting

The address bits generated by the ALPG, X_{n_0} (Y_{n_0} or Z_{n_0}), X_{n_1} (Y_{n_1} or Z_{n_1}) and X_{n_j} (Y_{n_j} or Z_{n_j}) are assigned to the scramble memory address bits, X_{m_0} , X_{m_1} , and X_{m_i} , respectively.

X, Y, and Z settings

The address bits generated by the ALPG, X_{n_j} , Y_{n_j} , and Z_{n_j} are assigned to the scramble memory address bits, X_{m_0} , X_{m_1} , and X_{m_i} , respectively.

The four types of settings described above refer to X address settings. They also apply to Y address settings.

These settings can be combined together under the conditions described below.

- The number of address bits written on the left side of each equation is equal to the number of address bits written on the right side.
- The subscripts in each equation do not exceed the maximum bit number of X, Y, and Z addresses for the ALPG in the test system.



Z can be written on the right side only when the prescrambler is used.

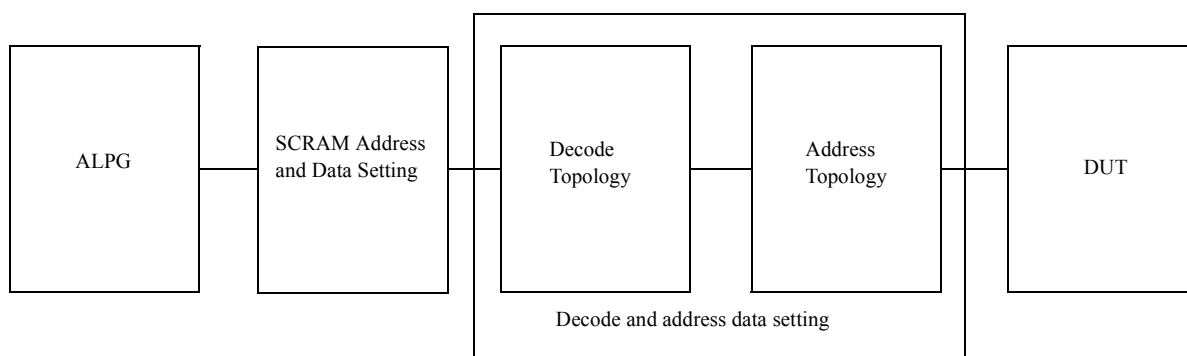
◆ Assigning ALPG Address Bit Y0 to Scramble Memory Address Bit X0

```
SCRAM EXAMPLE2
AMAX = #3FF, #3FF
ALINK
X0 = AY0
END
```

The above example shows that the scramble memory address bit X0 corresponds to the ALPG address bit Y0 and that other address bits (X1-11 and Y0-11) are not changed.

5. 3 Combined Settings of One-to-one and Logical Description

The SCRAM address/data setting (one-to-one) and decode/address data setting (with logical description) can be written in the same scramble program. Accordingly, two types of address conversion can be performed at the same time.



➔ For more information on each statement, refer to [5. 1 "Setting Decode and Address Topology \(In Logical Notation\)" on page 5-1](#) and [5. 2 "Setting Addresses and Data \(Individually\)" on page 5-12](#).

To combine one-to-one address and data settings with logical description, the REPLACE statement described below is required.

5. 3. 1 REPLACE

Function: The declaration statement used to combine one-to-one address and data settings with logical description.

Statements listed after REPLACE are used to assign data to X and Y addresses with other settings.

➔ For more information on how to set X and Y addresses and data, refer to [5. 2. 1 "X Address and Y Address/Data Setting Statement" on page 5-12](#).

Syntax

```
REPLACE
```

```
X,Y address/data setting statement
```

This statement can be written in a program only once. This statement must be specified between the AMAX statement and the EXPRESSION or ADDRESS statement.

This statement cannot be used in a program which is using the ALINK statement.

6. *Compilation Statement*

The revision number can be added to the object scramble program file through compilation.

6. 1 Adding Revision Information to the Object File

6. 1. 1 %REMARK Statement

Function: The character string between %REMARK and the carriage return are stored as the revision information in the object file.

Syntax

```
%REMARK Revision Information
```

Description

For the revision information character string, 0 to 9, a to z, A to Z, asterisks (*), underlines (_), periods (.), and dollar signs (\$) can be used. Revision information can be omitted. When more than 80 characters are specified, the 81st character and higher are ignored. When the %REMARK statement is written in a program more than once, the second and subsequent statements are ignored.

Appendix 1. Scramble Program Example

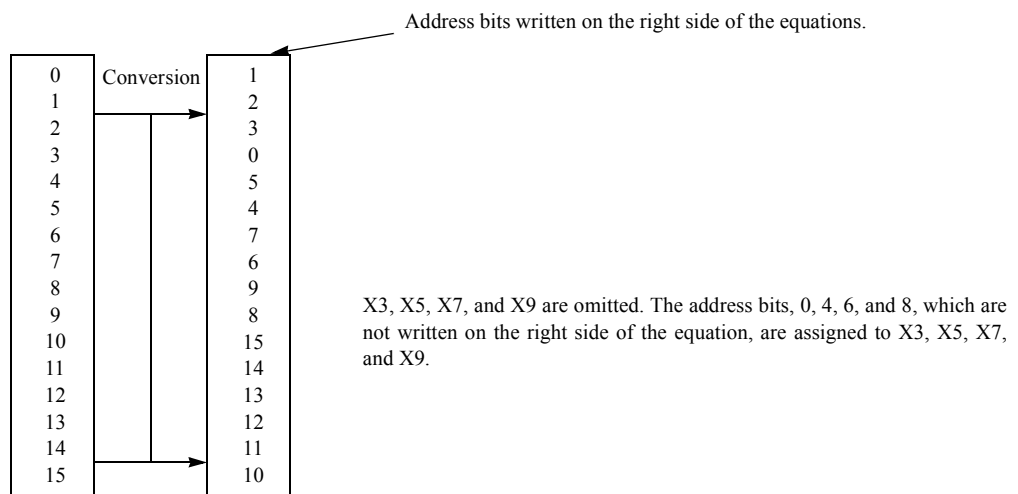
The example below shows address bit assignments.

```

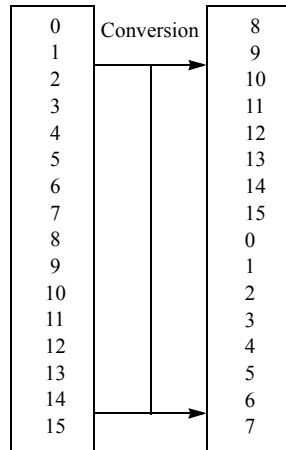
000001 00001  SCRAM SAMPLE           : Scramble program name
000002 00002
000003 00003          RDX 10          : Sets the radix to 10, which is used for
000004 00004          : constants.
000005 00005          AMAX=#F #F      : Sets the maximum X and Y addresses to #F.
000006 00006
000007 00007
000008 00008
000009 00009
000010 00010          X0,2,4,6,8,10-15=1,3,5,7,9,15-10 --- (a)
000011 00011          X1=2            --- (a)
000012 00012
000013 00013          Y0-7=8-15      --- (b)
000014 00014          Y8,9,10=0,1,2  --- (b)
000015 00015
000016 00016
000017 00017
000018 00018
000019 00019
000020 00020
000021 00021
000022 00022  END

```

(a) defines X address bits.



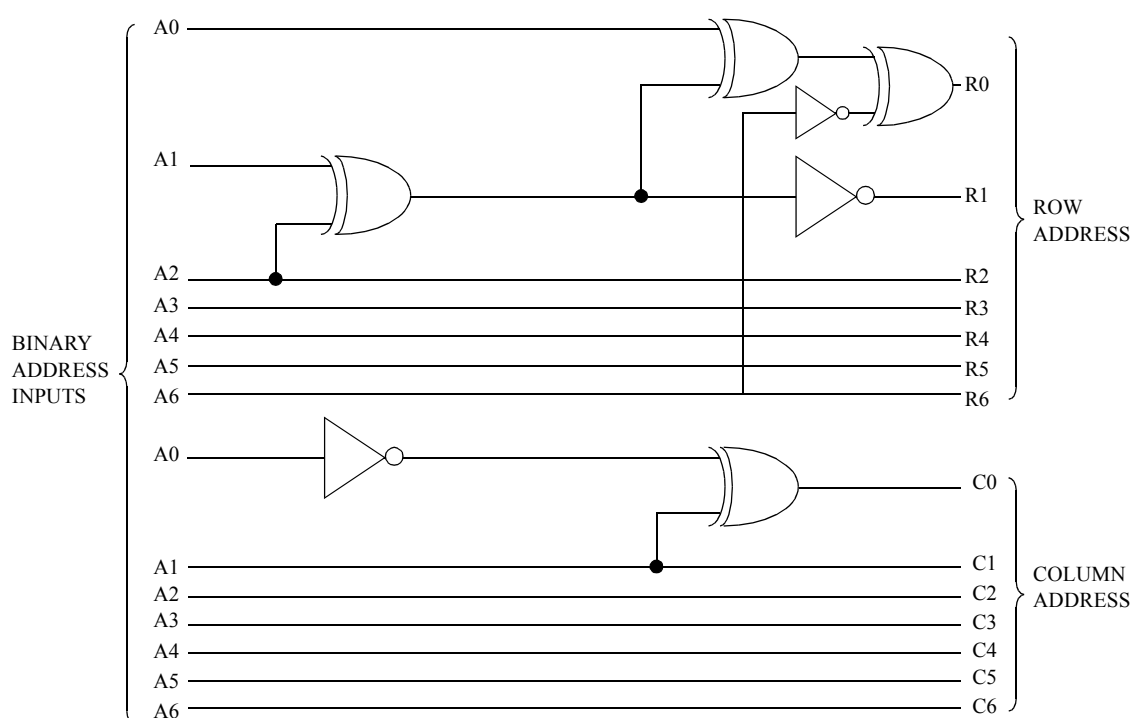
(b) defines Y address bits.



Appendix 2. Logical Description Program Example

The example below shows the program of logical description.

- MK4116 decode topology



The above figure can be expressed using the logical description shown below.

```

XA0 = X2.XOR.X1.XOR.X0.XOR..NOT.X6
XA1 = .NOT.(X1.XOR.X2)
XA2 = X2
XA3 = X3
XA4 = X4
XA5 = X5
XA6 = X6

YA0 = .NOT.Y0.XOR.Y1

```

- MK4116 address topology

TRANSFORMATION FROM DATA SHEET PIN
NAMES TO MK4116 INTERNAL PIN NAMES

PIN NUMBER	MK4116 DATA SHEET	MK4116 ACTUAL
13	A6	A0
10	A5	A1
11	A4	A2
12	A3	A3
7	A2	A4
6	A1	A5
5	A0	A6

The MK4116 address topology shown above is programmed as shown below.

X0 = XA6
X1 = XA5
X2 = XA4
X3 = XA3
X4 = XA2
X5 = XA1
X6 = XA0

Y0 = YA6
Y1 = YA5
Y2 = YA4
Y3 = YA3
Y4 = YA2
Y5 = YA1
Y6 = YA0

The actual scramble program for the MK4116 decode and address topology is shown below.

```
000001 00001  SCRAM SAMPLE2
000002 00002
000003 00003      AMAX=7F 7F
000004 00004
000005 00005  EXPRESSION
000006 00006      XA0=X2.XOR.X1.XOR.X0.XOR..NOT.X6
000007 00007      XA1=.NOT.(X1.XOR.X2)
000008 00008      XA2=X2
000009 00009      XA3=X3
000010 00010      XA4=X4
000011 00011      XA5=X5
000012 00012      XA6=X6
000013 00013
000014 00014      YA0=.NOT.Y0.XOR.Y1
000015 00015  ADDRESS
000016 00016      X0=XA6
000017 00017      X1=XA5
000018 00018      X2=XA4
000019 00019      X3=XA3
000020 00020      X4=XA2
000021 00021      X5=XA1
000022 00022      X6=XA0
000023 00023
000024 00024      Y0=YA6
000025 00025      Y1=YA5
000026 00026      Y2=YA4
000027 00027      Y3=YA3
000028 00028      Y4=YA2
000029 00029      Y5=YA1
000030 00030      Y6=YA0
000031 00031  END
```

When this scramble program is compiled with the command option -o, the scramble memory data is displayed. The scramble memory data is shown below.

X-ADDRESS

```

0000: 0060 0020 0000 0040 0010 0050 0070 0030
0008: 0068 0028 0008 0048 0018 0058 0078 0038
0010: 0064 0024 0004 0044 0014 0054 0074 0034
0018: 006C 002C 000C 004C 001C 005C 007C 003C
0020: 0062 0022 0002 0042 0012 0052 0072 0032
0028: 006A 002A 000A 004A 001A 005A 007A 003A
0030: 0066 0026 0006 0046 0016 0056 0076 0036
0038: 006E 002E 000E 004E 001E 005E 007E 003E

```

:

Y-ADDRESS

```

0000: 0040 0000 0020 0060 0050 0010 0030 0070
0008: 0048 0008 0028 0068 0058 0018 0038 0078
0010: 0044 0004 0024 0064 0054 0014 0034 0074
0018: 004C 000C 002C 006C 005C 001C 003C 007C
0020: 0042 0002 0022 0062 0052 0012 0032 0072
0028: 004A 000A 002A 006A 005A 001A 003A 007A
0030: 0046 0006 0026 0066 0056 0016 0036 0076
0038: 004E 000E 002E 006E 005E 001E 003E 007E

```

:

Z-ADDRESS

```

SCR SAMPLE2 ASSEMBLE OK!
AMAX=3F 3F 0

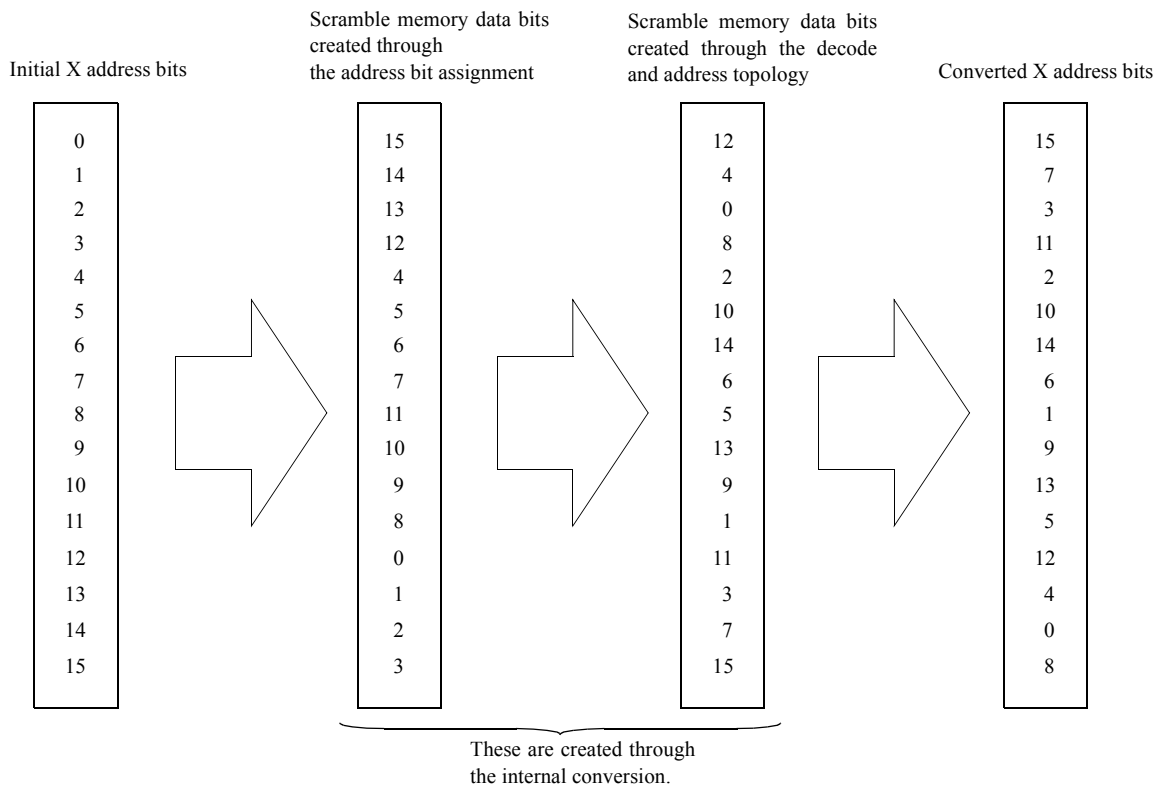
```

Appendix 3. Example Program which Uses One-to-one and Logical Description

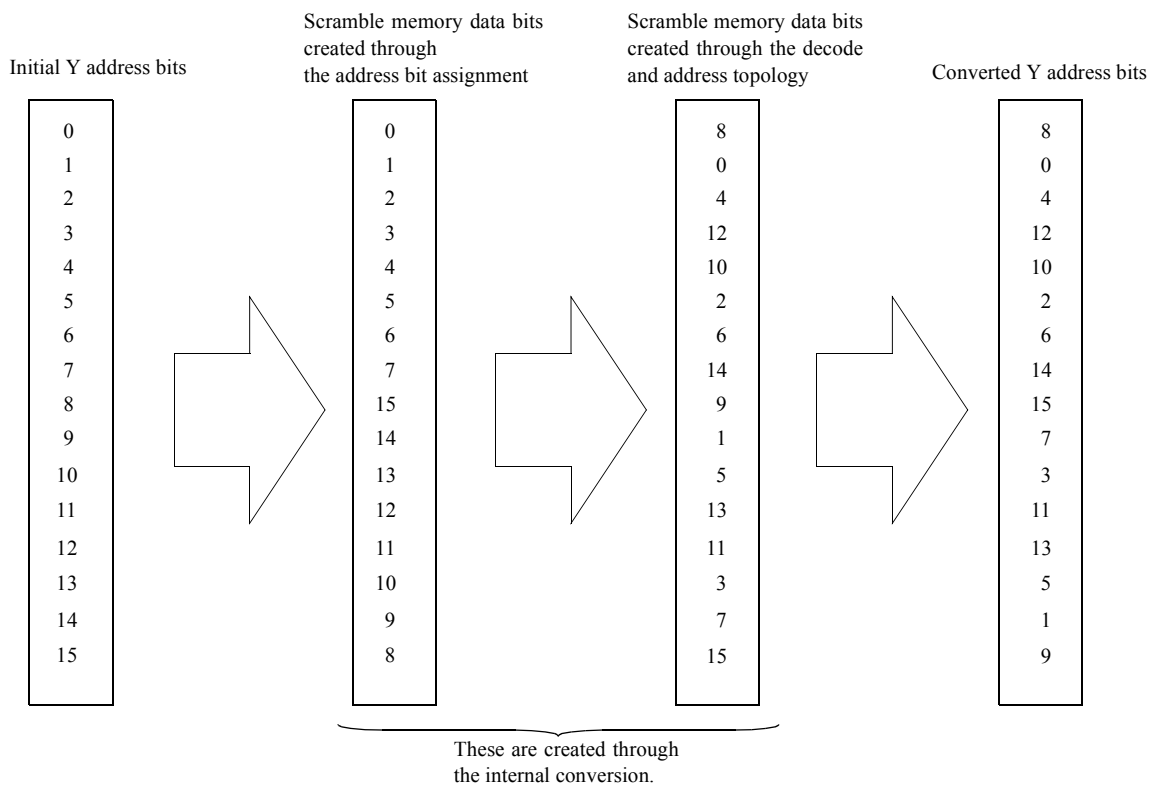
The example below shows a program including one-to-one and logical description.

000001	00001	SCRAM SAMPLE3	
000002	00002		
000003	00003	RDX 10	
000004	00004		
000005	00005	AMAX=#F #F	
000006	00006		
000007	00007		
000008	00008	REPLACE	
000009	00009	X0-3,8-15 = 15-12,11-8,0-3	] One-to-one description
000010	00010		
000011	00011	Y0-15 = 0-7,15-8	
000012	00012		
000013	00013		
000014	00014	EXPRESSION	
000015	00015	XA0=X2.XOR.X1.XOR.X0.XOR..NOT.X3	] Logical description
000016	00016	XA1=.NOT.(X1.XOR.X2)	
000017	00017	XA2=X2	
000018	00018	XA3=X3	
000019	00019		
000020	00020	YA0=.NOT.Y0.XOR.Y1	
000021	00021		
000022	00022	ADDRESS	
000023	00023	X0=XA3	
000024	00024	X1=XA2	
000025	00025	X2=XA1	
000026	00026	X3=XA0	
000027	00027		
000028	00028	Y0=YA3	
000029	00029	Y1=YA2	
000030	00030	Y2=YA1	
000031	00031	Y3=YA0	
000032	00032		
000033	00033	END	

Converting X addresses



Converting Y addresses



List of Figures

Figure 1-1	Decode Topology and Address Topology	1-1
Figure 1-2	Scramble Memory	1-1
Figure 5-1	X Address Decode Topology	5-3
Figure 5-2	Y Address Decode Topology	5-4

List of Tables

Table 5-1	Subscript Range	5-2
Table 5-2	Subscript Range	5-4

Index

[Symbols]

%REMARK..... 6-1

[A]

ADDRESS..... 5-9

ALINK..... 5-13

AMAX..... 4-2

ARI..... 4-6

ARIRAM..... 4-5

ARIRAM DEFINE..... 4-3

[D]

D..... 4-5

[E]

END..... 4-3

EXPRESSION..... 5-1

[I]

INV..... 4-5

[R]

RDX..... 4-1

REPLACE..... 5-16

[S]

SCRAM..... 4-1

[X]

X..... 5-12

XA..... 5-2

[Y]

Y..... 5-12

YA..... 5-2